Third-Party Access to Compiler State

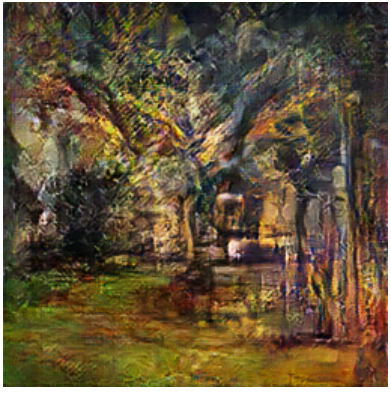
Why, How, Challenges

Alona Enraght-Moony

2026-05-19

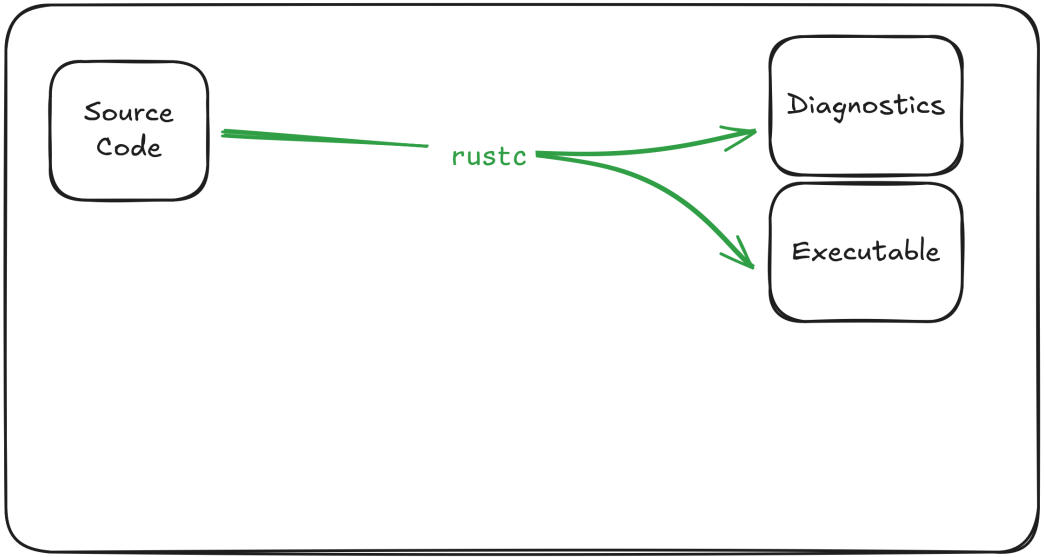
whoami(1)

- Alona Enraght-Moony (she/her)
- Rustdoc team since 2022
- Focused there on maintaining rustdoc-json
- This talk comes from soul-searching about the place that rustdoc-json fits into the ecosystem

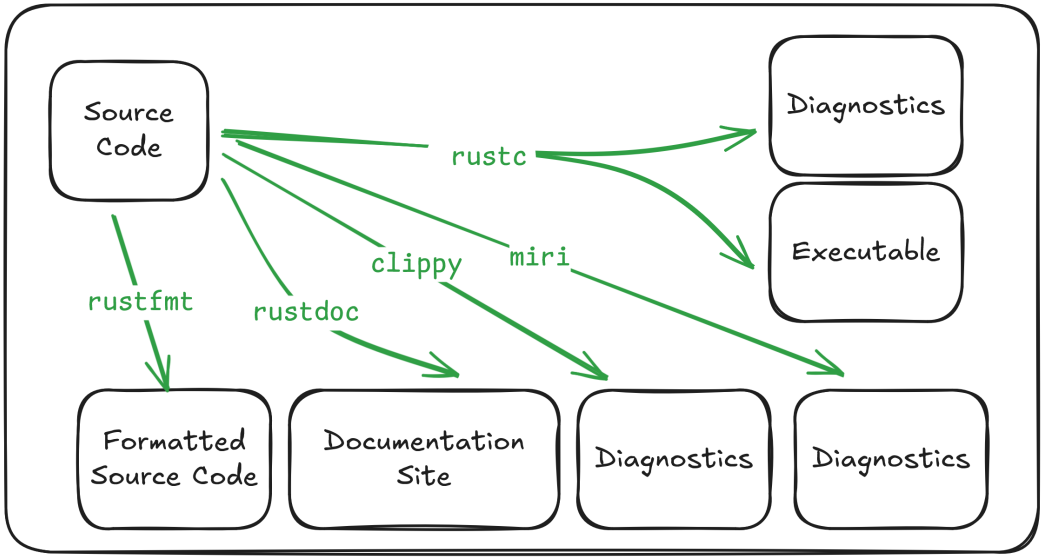


aDotInTheVoid

rustc, the 10'000 feet view



`rustc` isn't enough: Rust Project tools



~~rustc~~ the Rust Project isn't enough: Third party tools

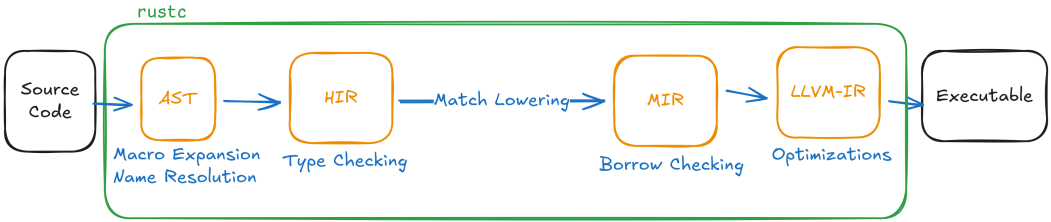
- Linters
  - cargo-check-external-types
  - cargo-semver-checks
  - klint
  - dylint
  - marker
- Verifiers
  - kani
  - aeneas
  - Creusot
- Refactoring tools
  - CoccinelleForRust
- Inspection
  - cargo-public-api
  - cargo-modules
- Custom codegen backends
  - cuda-oxide
  - rust-gpu
  - rust-cuda
- Others
  - Crubit
  - Charon
  - Minirust

Your tools here too!

Agenda

- How Rust Project tools work
- Why third-party tools can't work the same way
- Many approaches third-party tool can take

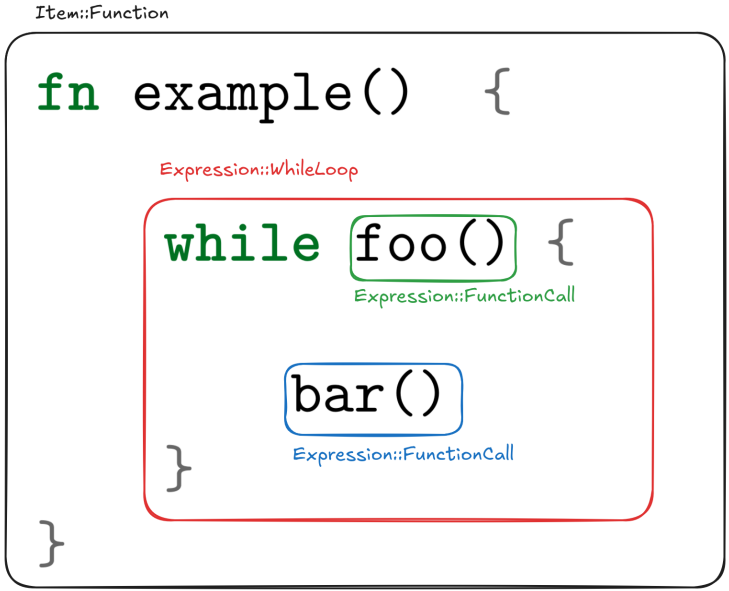
rustc, the 10'000 feet view



IR: Source Code

```
fn example()  {
    // Do bar until foo isn't true
    while foo() {
        bar()
    }
}
```

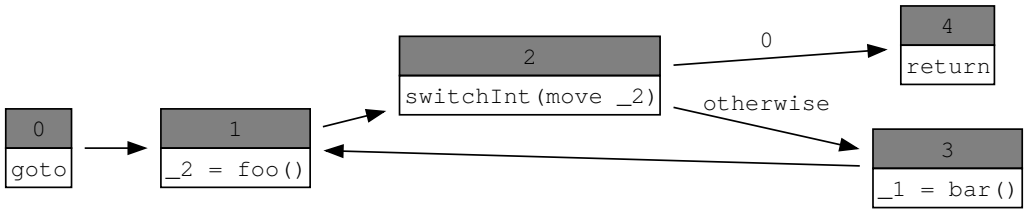
IR: AST


Item::Function
fn example() {
Expression::WhileLoop
while foo() {
Expression::FunctionCall
bar()
Expression::FunctionCall
}
}

IR: HIR

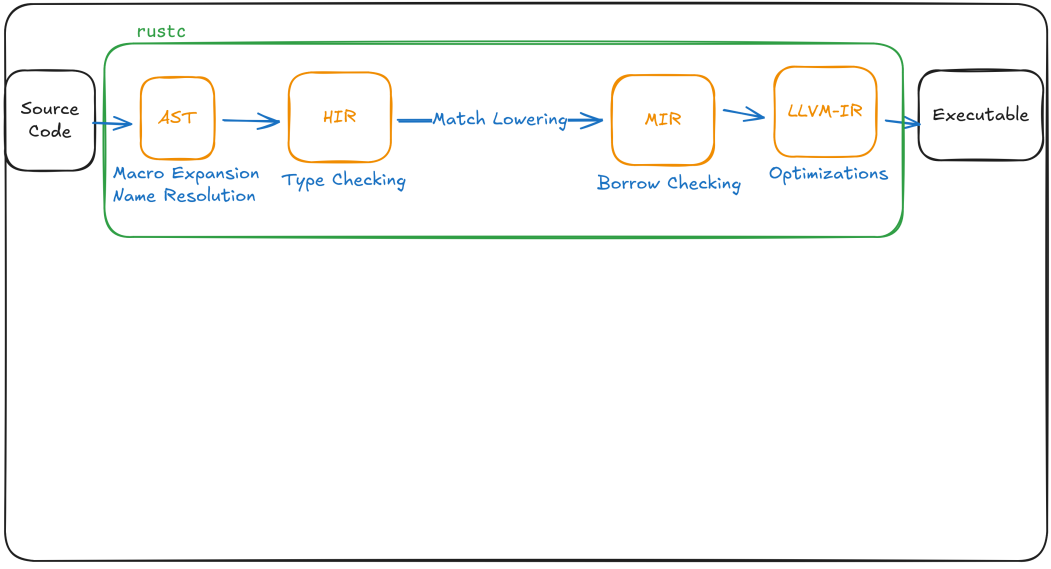
```
fn example() {
    loop {
        if foo() { // resolved
            bar() // resolved
        } else {
            break;
        }
    }
}
```

IR: MIR

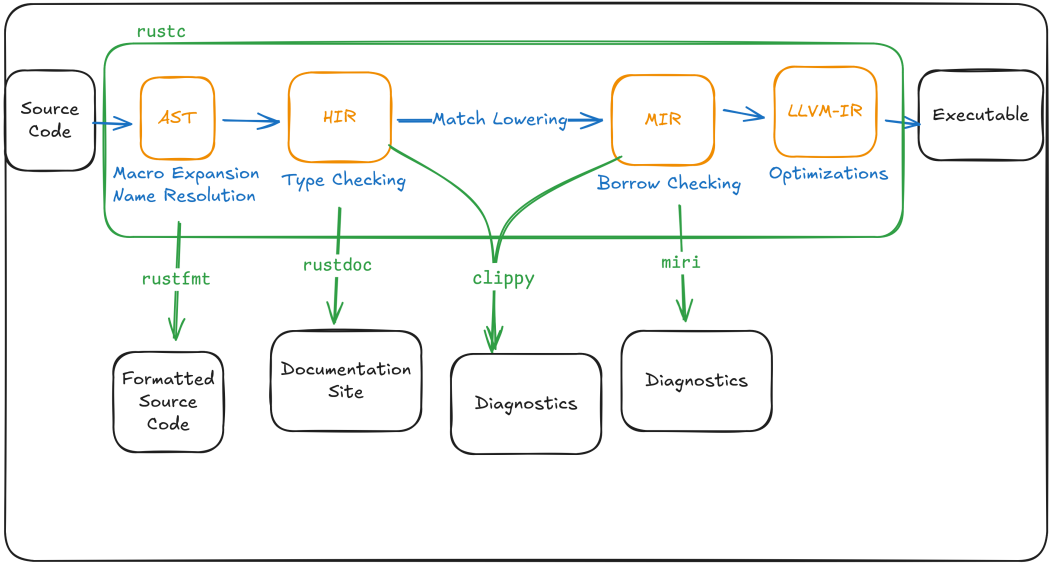



```
fn example() -> ()
let mut _1: ();
let mut _2: bool;
```

rustc, the 10'000 feet view



rustc, and the Rust Project tools



How tools call into `rustc`

```
extern crate rustc_interface; // And more

// set up target, flags, and dependencies
let config = ...;

rustc_interface::run_compiler(config, |compiler| {
    let ast = rustc_interface::passes::parse(&compiler.sess);
    rustc_interface::create_and_enter_global_ctxt(compiler, ast, |tcx| {
        // Real work done here
        tcx.get_whatever();
    })
});
```

Can third-party tools be like Rust Project tools?

```
#![feature(rustc_private)]

extern crate rustc_interface; // And more

// set up target, flags, and dependencies
let config = ...;

rustc_interface::run_compiler(config, |compiler| {
    let ast = rustc_interface::passes::parse(&compiler.sess);
    rustc_interface::create_and_enter_global_ctxt(compiler, ast, |tcx| {
        // Real work done here
        tcx.get_whatever();
    })
});
```

Can third-party tools be like Rust Project tools?

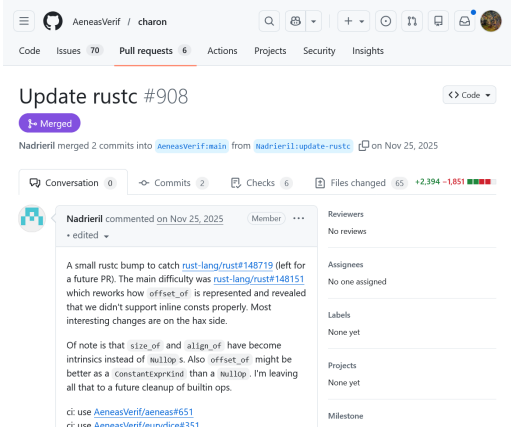
```
[toolchain]
channel = "nightly-2025-02-13"
components = ["llvm-tools-preview", "rustc-dev"]
```

Can third-party tools be like Rust Project tools?

```
$ ldd ./target/debug/demo_tool
        linux-vdso.so.1 (0x00007ffff7fd6000)
        librustc_driver-2fb444505ec98d04.so => /home/alona/.rustup/toolchains/nightly-2026-01-18-x86_64-unknown-li
nux-gnu/lib/librustc_driver-2fb444505ec98d04.so (0x00007d2cf0a00000)
        libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x00007d2cf86cd000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007d2cf0600000)
        libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007d2cf86c8000)
        libLLVM.so.21.1-rust-1.94.0-nightly => /home/alona/.rustup/toolchains/nightly-2026-01-18-x86_64-unknown-li
nux-gnu/lib/../lib/libLLVM.so.21.1-rust-1.94.0-nightly (0x00007d2ce6800000)
        librt.so.1 => /lib/x86_64-linux-gnu/librt.so.1 (0x00007d2cf86c1000)
        libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007d2cf86bc000)
        /lib64/ld-linux-x86-64.so.2 (0x00007d2cf8723000)
        libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007d2cf0909000)
        libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x00007d2cf869e000)
$
```

Problems for third-party tools

`rustc_private` APIs breaks frequently



Projects have died due to this:

- gazebo_lint
- semverver

`rustc_private` crates are unlike any other dependency

- `rustc` can't be build from crates.io with `cargo`, but needs its own build system
- The version of `rustc` that builds your tool is also the version of `rustc` your tool uses
- You need to be on nightly to use them
- That same version of nightly needs to be installed at runtime

How the Rust Project tools avoid this

`rustc_private` APIs breaks frequently

- Changes to `rustc` must make sure that `rustdoc`, `clippy`, `rustfmt`, `miri` tests still pass
- Changes to `rustc` APIs have to also change callers in Rust Project tools

`rustc_private` crates are unlike any other dependency

- First party tools are built with `x.py`, not `cargo`
- First party tools are distributed by `rustup`, not `cargo`

Both of these are possible because Rust Project tools are *developed, tested and distributed* alongside `rustc`

Summary: `#![feature(rustc_private)]`

Lets you directly access the compiler internals, the same way the compiler does.

- Pros:
  - Access to all the rich state `rustc` has
- Cons:
  - Large and cumbersome API
  - Very frequent breaking changes
  - Only usable on nightly
    - Both at build and run time
  - Need to dynamically link to specific version of `librustc_driver-<hash>.so`

`rustc_public`: A level of indirection

A wrapper crate implemented using `#![feature(rustc_private)]`

Currently distributed with `rustc` as an `extern crate`

- Pros:
  - More limited API, designed for tools
  - Less breaking changes
  - One day, will be on crates.io, support multiple `rustc` versions
- Cons:
  - Might not have the API you need, and need to fall back to `rustc_private` crates
  - Still need to link to version specific `librustc_driver.so`
  - crates.io and multi-version are still hypotheticals (as of today)

rust-analyzer: A compiler you can `cargo add`

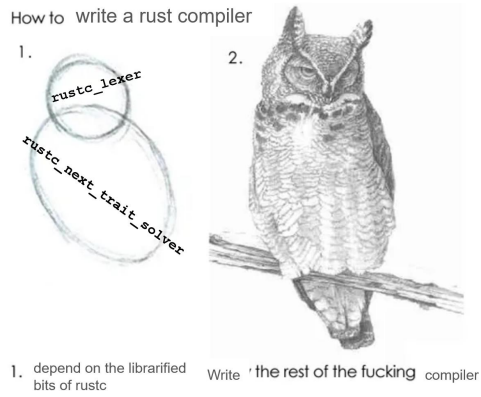
De nuevo implementation of a rust compiler, for IDE support

- Pros:
  - Use like any other library on crates.io
  - Works on stable
  - Produces freestanding binaries
- Cons:
  - Frequent breaking changes
  - Doesn't have a "complete" compiler:
    - No borrow-checking
  - Sometimes can give slightly different results to `rustc`
  - API designed for interactive IDE tooling:
    - Optimized for latency, not throughput

libraryification of `rustc`: Behind (bits of) rust-analyzer

Some bits of the compiler (e.g. `rustc_lexer` and `rustc_next_trait_solver`) are published on crates.io, initially to be used by rust-analyzer

- Pros:
  - Use like any other library on crates.io
  - Works on stable
  - Produces freestanding binaries
- Cons:
  - (Somewhat) frequent breaking changes
  - Only implements the "leaf"s of analysis, not the core:
    - Bring your own data-structures: `rustc_type_ir`
    - Draw the rest of the owl



`rustdoc --output-format=json`: The one I worked on

Rustdoc feature to emit an IR to a JSON file

- Pros:
  - Your tool talks to `rustc` over a json file, not in memory
  - Your tool can be in stable, but use nightly `rustc` at runtime
  - Small API
  - Infrequent breaking changes
- Cons:
  - Tied to `rustdoc`, not `rustc`: presentational representation, not semantics
  - Only describes API, nothing about the contents of functions
  - Can't request the information you need, get all the information whether you care or not

Charon: JSON Emiter but with function bodies

A third-party tool implemented with `#![feature(rustc_private)]` that emits an IR to a JSON file

- Pros:
  - Your tool talks to `rustc` over a json file, not in memory
  - Your tool can be in stable, but use nightly `rustc` at runtime
  - IR optimized for semantics and verification
- Cons:
  - Focused on semantics, not syntax
  - Charon itself requires specific nightly
  - Can't request the information you need, get all the information whether you care or not

~~50~~ 6 Ways

- Use the compiler crates in your process:
  - `rustc_private`
  - `rustc_public`
- Normal libraries on crates.io:
  - rust-analyzer
  - libraryification
- Emit IR to a file:
  - `rustdoc --output-format=json`
  - Charon

Summary

- Compilers to more that just take inputs and produce outputs:
  - Their intermediate state is the building block for tools
- Rust Project tools have convenient access to this state
- But third-party tools do not
- Many approaches to getting at it

That's all folks!

Slides online: https://alona.page/talks/third-party-access.pdf

Thanks

Jynn Nelson, Julia Ryan, David Barsky

Contact

- email: me@alona.page
- fedi: @adotinthevoid@hachyderm.io
- bluesky: alona.page